WeAct Studio RP2040 Core Board (16 MB) — Hardware Notes and Final Configuration

> **Status (2026-10-25).** This file replaces the earlier investigation notes. The first WeAct build failed with “JTAG scan chain interrogation failed: all ones”. The cause was neither the wiring nor the `DAP_SWJ_Pins` bit layout: the WeAct `CMakeLists.txt` had disabled the SDK stdio UART, which on the Pico build is what holds **nTRST (GP0) high from boot**. The fix (keep the stdio UART enabled, and drive nTRST high in `DAP_SETUP()`) is verified on hardware. Section 3 explains the mechanism.
> The complete configuration, build and Pico-vs-WeAct guide is `WEACT_CONFIGURATION_BUILD_AND_DIFFERENCES.md`.

1. Hardware: WeAct RP2040 16 MB vs Raspberry Pi Pico

Feature / Pin	Raspberry Pi Pico	WeAct Studio RP2040 V1.0 (16 MB)	Impact on the probe
QSPI flash (U2)	2 MB (`W25Q16JVUXIQ`; some variants 4 MB)	**16 MB** (W25Q128-class; the schematic footprint is W25Q32JVSS)	`PICO_BOARD=weact_studio_rp2040_16mb` sets `PICO_FLASH_SIZE_BYTES=16 MB`; the same `boot2_w25q080` and `PICO_FLASH_SPI_CLKDIV=2` as the Pico. With `copy_to_ram`, the flash is only read at boot.
Header pins 1–10 (GP0–GP7)	GP0–GP7 + GND (positions 3, 8)	**Identical** (`J1` pins 20..11)	The 10-way ribbon used with a Pico plugs in unchanged.
Status LED (GP25)	Green LED, active high	Blue `LED1`, active high via 1 kΩ `R7`; red `PWR-LED` on 3.3 V	`PROBE_USB_CONNECTED_LED=25`; steady on when idle, 10 Hz blink on TCK/SWCLK activity.
GPIO 23 (GP23)	Internal SMPS power-save (`PS`)	User **`KEY`** button (`K3`), 1 kΩ to GND, 1 kΩ pull-up	The firmware does not drive GP23; it stays an input.
RUN / reset	Header pin 30 (`RUN`)	Header pin 30 plus on-board **`NRST`** button (`K2`)	Reset the probe without unplugging USB.
3V3_EN	Header pin 37 (SMPS enable)	**Not connected** (`NC`)	The board uses an `SC662K-3.3` LDO (250 mA max) fed from VSYS.
GP24 / GP29	GP24 is VBUS sense (not on the header)	**GP24 and GP29 are on the right header**	The right header is shifted relative to the Pico; see below.
SWD debug header (`J2`)	3 pins: SWCLK, GND, SWDIO	4 pins: 3V3, SWDIO, SWCLK, GND	Do not plug a Pico SWD cable into `J2`: its GND would land on SWDIO and SWCLK on 3.3 V.
USB	micro-USB	USB-C (5.1 kΩ CC pull-downs)	None.
Size	21 × 51 mm	21.52 × 53 mm	2 mm overhang; the 2.54 mm headers still fit.

The right header is **not** position-compatible with the Pico. Only GND (positions 3, 8, 13, 18) and ADC_VREF (position 6) are in the same places; in particular the WeAct's 3.3 V is at position 4, while the Pico's position 4 is `3V3_EN` and its 3.3 V is at position 5 (which is GP24 on the WeAct). Do not use a Pico-shaped adapter to take 3.3 V from the right header.

1.1 Probe signals (left header `J1`, positions 1–10)

Position	GPIO	Signal	Direction / implementation
1	0	**nTRST**	SIO output, driven high at boot and when JTAG starts; not host-controllable
2	1	**nRESET**	Open-drain: input with pull-up (released) or driven low
3	—	GND	—
4	2	**SWCLK / TCK**	PIO side-set 0; SIO for bit-banged commands
5	3	**SWDIO / TMS**	PIO SET 0 in JTAG; SIO in SWD
6	4	**UART TX**	UART1 TX, 115200 8N1 default
7	5	**UART RX**	UART1 RX
8	—	GND	—
9	6	**TDI**	PIO OUT 0
10	7	**TDO**	PIO IN 0; SIO input with pull-up, input synchroniser bypassed
on-board	25	**Status LED**	SIO, 1 kHz timer IRQ on core 0

The schematic confirms that GP0–GP7 go directly to `J1` with no other on-board loads, and that the 16 MB QSPI flash is on the RP2040's dedicated QSPI pins, not on GP0–GP5.

2. Final firmware configuration

- `CMakeLists.txt` selects `PICO_BOARD=weact_studio_rp2040_16mb` and keeps the SDK stdio UART **enabled** (`PICO_STDIO_UART`, the default). Do not set it to `OFF`; this is the setting that holds nTRST high from boot.
- `include/DAP_config.h` — `DAP_SETUP()` additionally drives nTRST (GP0) high (latch first, then output enable) and enables the pull-up on nRESET (GP1).
- Pin numbers are unchanged from the working Pico build: nTRST GP0, nRESET GP1, TCK GP2, TMS GP3, UART1 GP4/GP5, TDI GP6, TDO GP7, LED GP25.

- `PICO_FLASH_SIZE_BYTES` is 16 MB; `copy_to_ram` means the image runs from SRAM and only boot2 reads the flash.

The delivered build uses xPack GCC 15.2.1 and Pico SDK 2.3.0. See `WEACT_CONFIGURATION_BUILD_AND_DIFFERENCES.md` for the full commands and the build-output sizes.

3. Why the first WeAct build failed (verified root cause)

3.1 Symptom

With the first WeAct build, OpenOCD connected to the probe but found no TAP:

```
Info : CMSIS-DAP: Interface Initialised (JTAG)
Info : SWCLK/TCK = 1 SWDIO/TMS = 1 TDI = 1 TDO = 1 nTRST = 1 nRESET = 1
Info : CMSIS-DAP: Interface ready
Info : clock speed 4000 kHz
Info : cmsis-dap JTAG TLR_RESET
Info : cmsis-dap JTAG TLR_RESET
Error: JTAG scan chain interrogation failed: all ones
Error: bcm2837.cpu: IR capture error; saw 0x0f not 0x01
```

The same cable, target and OpenOCD configuration worked with the Pico build.

3.2 Mechanism

`PIN_nTRST_OUT()` is an empty stub and `PIN_nTRST_IN()` returns 0, so the probe cannot let the host drive nTRST: GP0 is whatever the firmware's own GPIO code leaves on it.

The working Pico build holds GP0 high from boot by accident:

1. `PICO_STDIO_UART` defaults to ON, so TinyUSB's `board_init()` calls `stdio_uart_init_full(uart0, 115200, 0, 1)`.
2. Both the Pico and WeAct board headers put UART0 TX on **GP0** and RX on **GP1**.
3. UART0 TX idles **high**, so nTRST is released from `board_init()` until `pio_jtag_init()` later re-muxes GP0 to SIO (also high).

The first WeAct `CMakeLists.txt` contained:

```
set(PICO_STDIO_UART OFF CACHE BOOL "..." FORCE)
```

That removed the UART0 setup. On the WeAct, GP0 then stayed an input with the RP2040 pad reset pull-down from `DAP_SETUP()` until OpenOCD sent `DAP_Connect`. The target saw nTRST **asserted** for the whole boot/idle period; its TAP was in Test-Logic-Reset when the scan started, TDO stayed tri-stated, and the probe read 1 on every bit.

The PIO JTAG sequencer (TCK/TMS/TDI stream and TDO capture) was never at fault. That is why the probe's own pin activity looked correct on the bench while OpenOCD still saw all ones.

3.3 Fix

1. **`CMakeLists.txt`** — removed the `PICO_STDIO_UART OFF` override, so the build behaves like the working Pico build: UART0 on GP0/GP1, nTRST high from boot.
2. **`include/DAP_config.h`** — `DAP_SETUP()` drives GP0 high explicitly and releases nRESET with a pull-up, so the idle state no longer depends only on the stdio configuration.

3.4 Verification

After rebuilding with GCC 15.2.1, JTAG works with OpenOCD on the same target and cable as the Pico build, and the UART bridge works. The probe's `DAP_Setup` sets the GP0 latch and output enable high and calls `gpio_set_pulls(1, up=1, down=0)` for GP1.

4. Bench observations that were misleading

The earlier notes concluded from bench tests that GP0/GP7 were open-circuited or cold-soldered. Those conclusions were not supported:

- Driving `DAP_JTAG_Sequence` and reading back the probe's own pads showed TCK/TMS/TDI toggling correctly and TDO reading 1. That only proves the probe drives its outputs; a target held in reset leaves TDO tri-stated, and the probe's TDO pull-up reads 1.
- The "charge and release" test (drive a pin, switch to high-Z, read the pad) showed GP0/GP1 holding their level and GP7 at 1. A JTAG-enabled target may present weak or disabled pulls on TRST/TDO, and the probe's own pull-up dominates TDO; this is consistent with a working connection to a target in reset, not with an open circuit.
- The `SWCLK/TCK = ...` status line is **not** in CMSIS-DAP specification order because `DAP_SWJ_*` in `include/DAP.h` holds GPIO numbers (TCK 2, TMS 3, TDI 6, TDO 7, nTRST 0, nRESET 1) instead of the protocol bit positions (0, 1, 2, 3, 5, 7). Do not use that line to judge the physical nTRST level.
- A later log showing `nTRST = 1` came from a temporary local firmware that implemented nTRST and changed `DAP_SWJ_*`; those changes are not in the current source, and they did not by themselves fix the scan.

The "all ones" error is a target-side symptom: the target never drove TDO. The probe's own nTRST idle state is what the fix in section 3 corrects.

5. References

- `WEACT_CONFIGURATION_BUILD_AND_DIFFERENCES.md` — configuration, build, flashing, usage and the complete Pico-vs-WeAct comparison.
- `HDK/sch/WeAct-RP2040-V10 SchDoc.pdf`, `HDK/PINout.PNG`, `HDK/T.png`, `HDK/B.png` in `/home/dan/Downloads/PICO/WeActStudio.RP2040CoreBoard-main/`.
- Pico SDK 2.3.0: `/home/dan/PICO/pico-sdk`.
- CMSIS-DAP specification; OpenOCD 0.12 user's guide.